

ECM2433

Mock Paper 1

The C Family

Closed Book

Additional Materials: None

Permitted Materials: No calculators permitted

Guidelines:

The duration of this paper is 2hr 00.

Answer ALL questions.

Write all answers in the answer booklet provided.

Use only black pen or pencil.

Mobile and electronic devices are not allowed.

This is a mock paper in the style of the May paper. Possible answers are skeleton answers.

Answers and marking notes are at the end of this document.

Question 1

(a) Consider the following fragment. Explain what `fflush(stdout)` changes, and state one situation where the output before the newline might otherwise not appear immediately. (4 marks)

```
#include <stdio.h>

int main(void) {
    printf("loading");
    fflush(stdout);
    printf(" done\n");
    return 0;
}
```

(b) A project contains `main.c`, `stats.c`, and `stats.h`. Write a Makefile fragment with variables `CC` and `CFLAGS` that builds an executable named `report`. Include rules for `report`, `main.o`, `stats.o`, and `clean`. (6 marks)

(c) Complete the function so it returns a newly allocated copy of `src`. The caller will free the returned pointer. Handle allocation failure. (5 marks)

```
#include <stdlib.h>
#include <string.h>

char *copy_label(const char *src) {
    char *copy = /* allocate enough space */;
    if (copy == NULL) {
        return NULL;
    }
    /* copy the string */
    return copy;
}
```

(d) Complete the comparator and call `qsort` so the array is sorted by score descending. If two scores are equal, sort by name ascending. (6 marks)

```
#include <stdlib.h>
#include <string.h>

typedef struct {
    char name[16];
    int score;
} Entry;

int compare_entries(const void *left, const void *right) {
    /* complete this comparator */
}

void sort_entries(Entry entries[], size_t len) {
    /* call qsort */
}
```

(e) Explain the bug in the final `printf`, and name the kind of behaviour C gives for that access. (4 marks)

```
int *p = malloc(sizeof *p);
*p = 10;
free(p);
printf("%d\n", *p);
```

(Total 25 marks)

Question 2

(a) Write battery.hpp and battery.cpp for a class Battery with a private int percent, a constructor taking an int, set_percent, and get_percent. The setter should ignore values outside 0 to 100. Use an include guard and member function definitions in the source file. (7 marks)

(b) The following class owns a raw heap array. Identify two resource-management problems and give a safer correction using either the rule of three idea or a standard RAII object. (5 marks)

```
class Buffer {
    char *data;
public:
    Buffer(int size) {
        data = new char[size];
    }

    ~Buffer() = default;
};
```

(c) Fix the binary search bug, then write a separate sort call that orders the same values descending. (5 marks)

```
#include <algorithm>
#include <vector>

std::vector<int> values = {9, 1, 4, 7};
bool found = std::binary_search(values.begin(), values.end(), 4);
```

(d) Trace the shared ownership and explain whether the weak pointer can still be locked at the marked line. (4 marks)

```
#include <memory>
#include <iostream>

auto a = std::make_shared<int>(42);
std::weak_ptr<int> watch = a;
auto b = a;
a.reset();

if (auto p = watch.lock()) {
    std::cout << *p << "\n";
}
```

(e) In the catch line below, explain why the exception is caught by const reference and what error.what() provides. (4 marks)

```
try {
    throw std::invalid_argument("negative age");
} catch (const std::invalid_argument& error) {
    std::cerr << error.what() << "\n";
}
```

(Total 25 marks)

Question 3

(a) The function below fails to compile. Explain the role of the semicolon and fix the function. (4 marks)

```
fn double(n: i32) -> i32 {  
    n * 2;  
}
```

(b) Complete `classify` as a single if expression. Explain why the expression needs an else branch. (4 marks)

```
fn classify(score: u32) -> &'static str {  
    let result = /* pass if score >= 40, otherwise fail */;  
    result  
}
```

(c) This program tries to use a moved `String`. Explain the error and show two fixes: one using borrowing, and one using clone. (5 marks)

```
fn print_name(name: String) {  
    println!("{name}");  
}  
  
fn main() {  
    let user = String::from("Ada");  
    print_name(user);  
    println!("{user}");  
}
```

(d) Write the `sort_by` call that sorts readings by the floating-point value ascending and then by id descending. Explain why `partial_cmp` is used. (6 marks)

```
let mut readings = vec![(0.5, 2), (0.1, 9), (0.5, 1)];  
/* sort here */
```

(e) Complete `parse_count` so it trims the input, parses an `i32`, and returns parse errors to the caller without `unwrap`. (6 marks)

```
fn parse_count(text: &str) -> Result<i32, std::num::ParseIntError> {  
    let n = /* parse with ? */;  
    Ok(n)  
}
```

(Total 25 marks)

Question 4

(a) Complete the loop so four worker threads all send one value through the same channel. Explain why `drop(tx)` is needed before the receiving loop can finish. (7 marks)

```
use std::sync::mpsc;
use std::thread;

fn main() {
    let (tx, rx) = mpsc::channel();

    for id in 0..4 {
        /* spawn a worker that sends id * 10 */
    }

    drop(tx);

    for value in rx {
        println!("{value}");
    }
}
```

(b) Complete the shared counter update using `Arc<Mutex<i32>>`. Explain what lock returns and why each update is exclusive. (6 marks)

```
use std::sync::{Arc, Mutex};
use std::thread;

let counter = Arc::new(Mutex::new(0));
let mut handles = Vec::new();

for _ in 0..4 {
    let counter = Arc::clone(&counter);
    handles.push(thread::spawn(move || {
        /* increment the shared counter */
    })));
}

for handle in handles {
    handle.join().unwrap();
}
```

(c) Fix the generic function signature so it can compare two values and print the larger one using normal display formatting. (4 marks)

```
fn show_larger<T>(a: T, b: T) {
    let winner = if a > b { a } else { b };
    println!("{winner}");
}
```

(d) Fix the recursive enum so Rust knows its size, then explain why the indirection is needed. (4 marks)

```
enum List {
    Cons(i32, List),
    Nil,
}
```

(e) For the custom `Team` key below, add the derives needed for insertion and lookup in a `HashMap`. State why those traits are needed. (4 marks)

```
use std::collections::HashMap;

struct Team {
    name: String,
    division: u8,
}

let mut scores = HashMap::new();
scores.insert(Team { name: String::from("red"), division: 1 }, 12);
```

(Total 25 marks)

Answers and marking notes

These are skeleton answers. Equivalent correct code and reasoning should receive credit.

Question 1

(a) Question (4 marks): Consider the following fragment. Explain what `fflush(stdout)` changes, and state one situation where the output before the newline might otherwise not appear immediately.

Answer: `fflush(stdout)` forces pending buffered output on `stdout` to be written immediately. Without it, the text may remain in the buffer until newline, a full buffer, or normal program exit.

(b) Question (6 marks): A project contains `main.c`, `stats.c`, and `stats.h`. Write a Makefile fragment with variables `CC` and `CFLAGS` that builds an executable named `report`. Include rules for `report`, `main.o`, `stats.o`, and `clean`.

Answer: One suitable Makefile:

```
CC = gcc
CFLAGS = -std=c99 -Wall -Wextra

report: main.o stats.o
<TAB>$(CC) $(CFLAGS) main.o stats.o -o report

main.o: main.c stats.h
<TAB>$(CC) $(CFLAGS) -c main.c -o main.o

stats.o: stats.c stats.h
<TAB>$(CC) $(CFLAGS) -c stats.c -o stats.o

clean:
<TAB>rm -f *.o report
```

(c) Question (5 marks): Complete the function so it returns a newly allocated copy of `src`. The caller will free the returned pointer. Handle allocation failure.

Answer: Allocate `strlen(src) + 1` bytes, check `NULL`, then copy the string.

```
char *copy_label(const char *src) {
    char *copy = malloc(strlen(src) + 1);
    if (copy == NULL) {
        return NULL;
    }
    strcpy(copy, src);
    return copy;
}
```

(d) Question (6 marks): Complete the comparator and call `qsort` so the array is sorted by score descending. If two scores are equal, sort by name ascending.

Answer: Comparator and `qsort` call:

```
int compare_entries(const void *left, const void *right) {
    const Entry *a = left;
    const Entry *b = right;

    if (a->score < b->score) {
        return 1;
    }
    if (a->score > b->score) {
        return -1;
    }
    return strcmp(a->name, b->name);
}

void sort_entries(Entry entries[], size_t len) {
    qsort(entries, len, sizeof entries[0], compare_entries);
}
```

(e) Question (4 marks): Explain the bug in the final `printf`, and name the kind of behaviour C gives for that access.

Answer: After `free(p)`, `p` is dangling. Dereferencing it is use-after-free and gives undefined behaviour.

Question 2

(a) Question (7 marks): Write `battery.hpp` and `battery.cpp` for a class `Battery` with a private `int` `percent`, a constructor taking an `int`, `set_percent`, and `get_percent`. The setter should ignore values outside 0 to 100. Use an include guard and member function definitions in the source file.

Answer: One correct split:

```
/* battery.hpp */
```

```

#ifndef BATTERY_HPP
#define BATTERY_HPP

class Battery {
    int percent;
public:
    Battery(int percent);
    void set_percent(int percent);
    int get_percent() const;
};

#endif

/* battery.cpp */
#include "battery.hpp"

Battery::Battery(int percent) : percent(0) {
    set_percent(percent);
}

void Battery::set_percent(int percent) {
    if (percent < 0 || percent > 100) {
        return;
    }
    this->percent = percent;
}

int Battery::get_percent() const {
    return percent;
}

```

(b) Question (5 marks): The following class owns a raw heap array. Identify two resource-management problems and give a safer correction using either the rule of three idea or a standard RAII object.

Answer: The default destructor does not delete the owned array, so it leaks. The default copy behaviour would also copy only the pointer, risking double delete if a destructor were added. Safer fixes include using `std::vector<char>` or `std::string`, or writing destructor, copy constructor, and copy assignment correctly.

```

#include <vector>

class Buffer {
    std::vector<char> data;
public:
    explicit Buffer(int size) : data(size) {}
};

```

(c) Question (5 marks): Fix the binary search bug, then write a separate sort call that orders the same values descending.

Answer: `binary_search` requires a sorted range. Then descending sort uses a comparator.

```

std::sort(values.begin(), values.end());
bool found = std::binary_search(values.begin(), values.end(), 4);

std::sort(values.begin(), values.end(), [](int a, int b) {
    return a > b;
});

```

(d) Question (4 marks): Trace the shared ownership and explain whether the weak pointer can still be locked at the marked line.

Answer: After `auto b = a`, there are two shared owners. `a.reset()` removes one owner, but `b` still owns the int. `watch.lock()` succeeds and prints 42. The object is destroyed when the last shared_ptr owner is gone.

(e) Question (4 marks): In the catch line below, explain why the exception is caught by const reference and what `error.what()` provides.

Answer: Catching by const reference avoids copying and preserves the dynamic exception object. `error.what()` returns a C string describing the exception, here negative age.

Question 3

(a) Question (4 marks): The function below fails to compile. Explain the role of the semicolon and fix the function.

Answer: The semicolon makes `n * 2` a statement, so the block evaluates to unit (). Remove the semicolon:

```

fn double(n: i32) -> i32 {
    n * 2
}

```

(b) Question (4 marks): Complete `classify` as a single if expression. Explain why the expression needs an else branch.

Answer: `if` is being used as an expression, so both branches must produce a value:

```
fn classify(score: u32) -> &'static str {
    let result = if score >= 40 {
        "pass"
    } else {
        "fail"
    };
    result
}
```

(c) Question (5 marks): This program tries to use a moved String. Explain the error and show two fixes: one using borrowing, and one using clone.

Answer: print_name takes ownership of the String, so user cannot be used afterwards. Borrowing fix:

```
fn print_name(name: &str) {
    println!("{name}");
}

let user = String::from("Ada");
print_name(&user);
println!("{user}");
```

Keeping the original print_name(String), clone fix:

```
let user = String::from("Ada");
print_name(user.clone());
println!("{user}");
```

(d) Question (6 marks): Write the sort_by call that sorts readings by the floating-point value ascending and then by id descending. Explain why partial_cmp is used.

Answer: Float sorting uses partial_cmp because NaN means floats may not have a total ordering.

```
readings.sort_by(|a, b| {
    a.0.partial_cmp(&b.0)
        .unwrap()
        .then_with(|| b.1.cmp(&a.1))
});
```

(e) Question (6 marks): Complete parse_count so it trims the input, parses an i32, and returns parse errors to the caller without unwrap.

Answer: Use parse with an explicit target type and ?. The function must return Result.

```
fn parse_count(text: &str) -> Result<i32, std::num::ParseIntError> {
    let n = text.trim().parse::<i32>()?;
    Ok(n)
}
```

Question 4

(a) Question (7 marks): Complete the loop so four worker threads all send one value through the same channel. Explain why drop(tx) is needed before the receiving loop can finish.

Answer: Clone the transmitter for each worker and move that clone into the closure. drop(tx) removes the original sender so the receiver loop can end once worker senders are gone.

```
for id in 0..4 {
    let tx = tx.clone();
    thread::spawn(move || {
        tx.send(id * 10).unwrap();
    });
}
```

(b) Question (6 marks): Complete the shared counter update using Arc<Mutex<i32>>. Explain what lock returns and why each update is exclusive.

Answer: Arc gives shared ownership; Mutex gives exclusive mutable access. lock returns a Result containing a guard. The guard dereferences to the protected value and unlocks when dropped.

```
handles.push(thread::spawn(move || {
    let mut value = counter.lock().unwrap();
    *value += 1;
})));
```

(c) Question (4 marks): Fix the generic function signature so it can compare two values and print the larger one using normal display formatting.

Answer: PartialOrd is needed for > and Display is needed for normal braces formatting.

```
use std::fmt::Display;

fn show_larger<T>(a: T, b: T)
where
    T: PartialOrd + Display,
```

```

{
    let winner = if a > b { a } else { b };
    println!("{winner}");
}

```

(d) Question (4 marks): Fix the recursive enum so Rust knows its size, then explain why the indirection is needed.

Answer: Box gives a fixed-size pointer to the rest of the list. Without indirection, List would contain another full List recursively and have no known size.

```

enum List {
    Cons(i32, Box<List>),
    Nil,
}

```

(e) Question (4 marks): For the custom Team key below, add the derives needed for insertion and lookup in a HashMap. State why those traits are needed.

Answer: HashMap keys need equality and hashing. Debug is often useful for tests but not required for HashMap itself.

```

#[derive(PartialEq, Eq, Hash)]
struct Team {
    name: String,
    division: u8,
}

```